

TP 3G & LTE

Groupe AR-2 : Dimitri Julmy, Dylan Flotte
Computer Science
HES-SO Master

11 mars 2026

Table des matières

1	Code Python	3
1.1	config.py	3
1.2	crypto.py	3
1.3	session.py	6
1.4	server.py	10
1.5	client.py	12
2	Capture du fonctionnement	15

Introduction

Le présent rapport expose les travaux réalisés dans le cadre du laboratoire "TP4 - 3G & LTE", effectué pour le module TSM - Advanced Communication Architecture. L'objectif principal de cette manipulation est de comprendre et d'implémenter les mécanismes d'authentification mutuelle introduits dans les réseaux 3G (UMTS) et perfectionnés dans les réseaux 4G (LTE), en développant un système client-serveur simulant les protocoles de sécurité bidirectionnels.

Les étapes essentielles de l'implémentation sont suivies, depuis la génération du challenge aléatoire (RAND) jusqu'à l'établissement d'une session sécurisée avec authentification mutuelle des deux parties. Ce travail reprend les fondements du GSM tout en introduisant une amélioration majeure : l'authentification du réseau par le terminal mobile via un token cryptographique. L'architecture développée permet ainsi de pallier les vulnérabilités du GSM en garantissant l'identité du serveur, prévenant ainsi les attaques de type IMSI catcher et les faux relais.

Ce laboratoire offre une approche pratique de l'évolution des mécanismes de sécurité entre les générations de réseaux mobiles et permet d'appréhender les améliorations successives introduites par les standards 3G et 4G en matière d'authentification et de protection de la confidentialité des communications.

Chapitre 1

Code Python

1.1 config.py

```
# Paramètres de connexion
HOST = "127.0.0.1"
PORT = 5000

# Clé secrète Ki (stockée dans USIM et AuC)
# En production, cette clé devrait être stockée de manière sécurisée
Ki = b"SuperSecretKey123"

# Paramètres de debug
DEBUG = True
```

1.2 crypto.py

```
import os
import hashlib
from config import DEBUG

def generate_rand():
    """
    Génère un nombre aléatoire RAND de 16 octets
    Utilisé par le serveur pour le challenge d'authentification
    """
    rand = os.urandom(16)
```

```
if DEBUG:
    print(f"[CRYPTO] RAND généré: {rand.hex()}")
return rand

def compute_sres(rand, ki):
    """
    Calcule la réponse signée SRES à partir de RAND et Ki
    SRES = premiers 4 octets de SHA256(RAND || Ki)

    Args:
        rand: Le challenge RAND (16 octets)
        ki: La clé secrète Ki

    Returns:
        SRES (4 octets)
    """
    sres = hashlib.sha256(rand + ki).digest()[:4]
    if DEBUG:
        print(f"[CRYPTO] SRES calculé: {sres.hex()}")
    return sres

def derive_kc(rand, ki):
    """
    Dérive la clé de session Kc à partir de RAND et Ki
    Kc = SHA256(Ki || RAND)

    Args:
        rand: Le challenge RAND (16 octets)
        ki: La clé secrète Ki

    Returns:
        Kc (32 octets)
    """
    kc = hashlib.sha256(ki + rand).digest()
    if DEBUG:
        print(f"[CRYPTO] Clé de session Kc dérivée: {kc.hex()}")
    return kc
```

```
def xor_cipher(data, key):
    """
    Chiffrement/déchiffrement XOR avec clé

    Args:
        data: Données à chiffrer/déchiffrer
        key: Clé de chiffrement

    Returns:
        Données chiffrées/déchiffrées
    """
    result = bytes([data[i] ^ key[i % len(key)] for i in range(len(data))])
    if DEBUG:
        print(f"[CRYPTO] XOR cipher appliqué - Taille: {len(data)} octets")
    return result


def compute_auth_token(rand, ki):
    """
    Calcule le token d'authentification du serveur
    Token = SHA256(RAND || Ki)

    Args:
        rand: Le challenge RAND
        ki: La clé secrète Ki

    Returns:
        Token d'authentification (32 octets)
    """
    token = hashlib.sha256(rand + ki).digest()
    if DEBUG:
        print(f"[CRYPTO] Token d'authentification calculé: {token.hex()}")
    return token
```

1.3 session.py

```
from config import Ki, DEBUG
from crypto import compute_sres, derive_kc, xor_cipher, compute_auth_token

class Session:
    """
    Classe gérant l'état d'une session d'authentification
    """

    def __init__(self):
        self.rand = None
        self.kc = None
        self.authenticated = False
        self.server_authenticated = False
        if DEBUG:
            print("[SESSION] Session initialisée")

    def set_rand(self, rand):
        """
        Enregistre le RAND reçu/généré
        """
        self.rand = rand
        if DEBUG:
            print(f"[SESSION] RAND enregistré: {rand.hex()}")

    def authenticate_client(self, received_sres):
        """
        Authentifie le client en vérifiant le SRES reçu

        Args:
            received_sres: SRES reçu du client

        Returns:
            True si authentifié, False sinon
        """
        if self.rand is None:
            if DEBUG:
```

```
        print("[SESSION] Erreur: RAND non défini")
    return False

expected_sres = compute_sres(self.rand, Ki)
self.authenticated = (received_sres == expected_sres)

if DEBUG:
    if self.authenticated:
        print("[SESSION] Client authentifié avec succès")
    else:
        print("[SESSION] Échec d'authentification du client")
        print(f"[SESSION] SRES attendu: {expected_sres.hex()}")
        print(f"[SESSION] SRES reçu: {received_sres.hex()}")

return self.authenticated

def authenticate_server(self, received_token):
    """
    Authentifie le serveur en vérifiant le token reçu

    Args:
        received_token: Token reçu du serveur

    Returns:
        True si authentifié, False sinon
    """
    if self.rand is None:
        if DEBUG:
            print("[SESSION] Erreur: RAND non défini")
        return False

    expected_token = compute_auth_token(self.rand, Ki)
    self.server_authenticated = (received_token == expected_token)

    if DEBUG:
        if self.server_authenticated:
            print("[SESSION] Serveur authentifié avec succès")
        else:
            print("[SESSION] Échec d'authentification du serveur")
```

```
        print(f"[SESSION]    Token attendu: {expected_token.hex()}")
        print(f"[SESSION]    Token reçu: {received_token.hex()}")

    return self.server_authenticated

def establish_session_key(self):
    """
    Établit la clé de session Kc

    Returns:
        La clé de session Kc
    """
    if self.rand is None:
        if DEBUG:
            print("[SESSION] Erreur: RAND non défini")
        return None

    self.kc = derive_kc(self.rand, Ki)
    if DEBUG:
        print(f"[SESSION] Clé de session établie: {self.kc.hex()}")
    return self.kc

def encrypt_data(self, data):
    """
    Chiffre des données avec la clé de session

    Args:
        data: Données à chiffrer

    Returns:
        Données chiffrées
    """
    if self.kc is None:
        if DEBUG:
            print("[SESSION] Erreur: Clé de session non établie")
        return None

    if DEBUG:
        print(f"[SESSION] Chiffrement de {len(data)} octets")
```

```
        return xor_cipher(data, self.kc)

def decrypt_data(self, cipher_data):
    """
    Déchiffre des données avec la clé de session

    Args:
        cipher_data: Données chiffrées

    Returns:
        Données déchiffrées
    """
    if self.kc is None:
        if DEBUG:
            print("[SESSION] Erreur: Clé de session non établie")
        return None

    if DEBUG:
        print(f"[SESSION] Déchiffrement de {len(cipher_data)} octets")

    return xor_cipher(cipher_data, self.kc)

def get_auth_token(self):
    """
    Génère le token d'authentification du serveur

    Returns:
        Token d'authentification
    """
    if self.rand is None:
        if DEBUG:
            print("[SESSION] Erreur: RAND non défini")
        return None

    return compute_auth_token(self.rand, Ki)

def is_authenticated(self):
    """
```

```
Vérifie si la session est authentifiée

Returns:
    True si authentifié, False sinon
"""
return self.authenticated

def is_server_authenticated(self):
    """
    Vérifie si le serveur est authentifié

    Returns:
        True si authentifié, False sinon
    """
    return self.server_authenticated
```

1.4 server.py

```
import socket
from config import HOST, PORT, DEBUG
from crypto import generate_rand
from session import Session

def main():
    # Initialisation de la session
    session = Session()

    # Création du socket serveur
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(1)

    print(f"[SERVEUR] En attente de connexion sur {HOST}:{PORT}...")
    print()

    conn, addr = server.accept()
    print(f"[SERVEUR] Client connecté depuis: {addr}")
    print()
```

```
try:
    # ÉTAPE 1: Génération et envoi du RAND
    RAND = generate_rand()
    session.set_rand(RAND)
    conn.send(RAND)
    print(f"[SERVEUR] RAND envoyé au client: {RAND.hex()}")
    print()

    # ÉTAPE 2: Réception et vérification du SRES
    print("[SERVEUR] Attente de la réponse SRES du client...")
    client_sres = conn.recv(1024)
    print(f"[SERVEUR] SRES reçu: {client_sres.hex()}")

    if not session.authenticate_client(client_sres):
        print("[SERVEUR] ÉCHEC: Authentification du client échouée")
        conn.close()
        server.close()
        return

    print("[SERVEUR] Client authentifié avec succès")
    print()

    # ÉTAPE 3: Authentification du serveur auprès du client
    token = session.get_auth_token()
    conn.send(token)
    print(f"[SERVEUR] Token d'authentification envoyé: {token.hex()}")
    print()

    # ÉTAPE 4: Établissement de la clé de session
    Kc = session.establish_session_key()
    print(f"[SERVEUR] Clé de session Kc établie: {Kc.hex()}")
    print()

    # ÉTAPE 5: Réception des données chiffrées
    print("[SERVEUR] Attente des données du client...")
    cipher_data = conn.recv(1024)
    print(f"[SERVEUR] Données chiffrées reçues ({len(cipher_data)} octets): {cipher_data.hex()}")
```

```
plain = session.decrypt_data(cipher_data)
print(f"[SERVEUR] Données déchiffrées: {plain}")
print()

# ÉTAPE 6: Envoi de la réponse chiffrée
message = b"Accuse de reception"
print(f"[SERVEUR] Message à envoyer: {message}")
cipher = session.encrypt_data(message)
print(f"[SERVEUR] Message chiffré: {cipher.hex()}")
conn.send(cipher)
print("[SERVEUR] Réponse envoyée au client")
print()

except Exception as e:
    print(f"[SERVEUR] Erreur: {e}")
finally:
    conn.close()
    server.close()
    print("[SERVEUR] Connexion fermée")

if __name__ == "__main__":
    main()
```

1.5 client.py

```
import socket
from config import HOST, PORT, DEBUG
from crypto import compute_sres
from session import Session

def main():
    # Initialisation de la session
    session = Session()

    # Connexion au serveur
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    print(f"[CLIENT] Connexion au serveur {HOST}:{PORT}...")
```

```
try:
    client.connect((HOST, PORT))
    print("[CLIENT] Connecté au serveur")
    print()

    # ÉTAPE 1: Réception du RAND
    print("[CLIENT] Attente du RAND du serveur...")
    RAND = client.recv(1024)
    session.set_rand(RAND)
    print(f"[CLIENT] RAND reçu: {RAND.hex()}")
    print()

    # ÉTAPE 2: Calcul et envoi du SRES
    print("[CLIENT] Calcul du SRES...")
    from config import Ki
    SRES = compute_sres(RAND, Ki)
    print(f"[CLIENT] SRES calculé: {SRES.hex()}")
    client.send(SRES)
    print("[CLIENT] SRES envoyé au serveur")
    print()

    # ÉTAPE 3: Authentification du serveur
    print("[CLIENT] Attente du token d'authentification...")
    token = client.recv(1024)
    print(f"[CLIENT] Token reçu: {token.hex()}")

    if not session.authenticate_server(token):
        print("[CLIENT] ÉCHEC: Serveur non authentifié")
        client.close()
        return

    print("[CLIENT] Serveur authentifié avec succès")
    print()

    # ÉTAPE 4: Établissement de la clé de session
    Kc = session.establish_session_key()
    print(f"[CLIENT] Clé de session Kc établie: {Kc.hex()}")
    print()
```

```
# ÉTAPE 5: Envoi de données chiffrées
# Paquet de données (~256 bits)
data = b"a3f7b2c1d9e845607f2c1a3b9d4e6f8a01c3b5d7e9f2a4c6b8d0e1f3a5c7b9d0"
print(f"[CLIENT] Données à envoyer ({len(data)} octets): {data}")

cipher = session.encrypt_data(data)
print(f"[CLIENT] Données chiffrées: {cipher.hex()}")
client.send(cipher)
print("[CLIENT] Données envoyées au serveur")
print()

# ÉTAPE 6: Réception de la réponse
print("[CLIENT] Attente de la réponse du serveur...")
cipher_resp = client.recv(1024)
print(f"[CLIENT] Réponse chiffrée reçue: {cipher_resp.hex()}")

resp = session.decrypt_data(cipher_resp)
print(f"[CLIENT] Réponse déchiffrée: {resp}")
print()

except Exception as e:
    print(f"[CLIENT] Erreur: {e}")
finally:
    client.close()
    print("[CLIENT] Connexion fermée")

if __name__ == "__main__":
    main()
```

Chapitre 2

Capture du fonctionnement

```
[SESSION] Session initialisée
[SERVEUR] En attente de connexion sur 127.0.0.1:5000...

[SERVEUR] Client connecté depuis: ('127.0.0.1', 48060)

[CRYPTO] RAND généré: 264cabdd49f335d16f8a2918fe0d18f76
[SESSION] RAND enregistré: 264cabdd49f335d16f8a2918fe0d18f76
[SERVEUR] RAND envoyé au client: 264cabdd49f335d16f8a2918fe0d18f76

[SERVEUR] Attente de la réponse SRES du client...
[SERVEUR] SRES reçu: c5314078
[CRYPTO] SRES calculé: c5314078
[SESSION] Client authentifié avec succès
[SERVEUR] Client authentifié avec succès

[CRYPTO] Token d'authentification calculé: c5314078ea3ecf1c027ec38093c6f721bba3fb18220f425cb6fd08bdd598c57c
[SERVEUR] Token d'authentification envoyé: c5314078ea3ecf1c027ec38093c6f721bba3fb18220f425cb6fd08bdd598c57c

[CRYPTO] Clé de session Kc dérivée: 036be1ca7c9683eb5bf1de2e2b6abf6b89f219dea7b08f26c8c8edab182b36e6
[SESSION] Clé de session établie: 036be1ca7c9683eb5bf1de2e2b6abf6b89f219dea7b08f26c8c8edab182b36e6
[SERVEUR] Clé de session Kc établie: 036be1ca7c9683eb5bf1de2e2b6abf6b89f219dea7b08f26c8c8edab182b36e6

[SERVEUR] Attente des données du client...
[SERVEUR] Données chiffrées reçues (64 octets): 625887fd1ea4e0da3fc8bb161f5f895bbe942bbd96d1bc44f1acd9ce2e4d0e87335a82f91ea3e7dc3ec8b81c4a5edc5debca7deec281e915a9fd8e9c7a1252d6
[SESSION] Déchiffrement de 64 octets
[CRYPTO] XOR cipher appliqué - Taille: 64 octets
[SERVEUR] Données déchiffrées: b'a3f7b2c1d9e845607f2cia3b9d4e6f8a01c3b5d7e9f2a4c6b8d0e1f3a5c7b9d0'

[SERVEUR] Message à envoyer: b'Accuse de reception'
[SESSION] Chiffrement de 19 octets
[CRYPTO] XOR cipher appliqué - Taille: 19 octets
[SERVEUR] Message chiffré: 420882bf0ff3a38f3ed1ac4b480fcf1fe09d77
[SERVEUR] Réponse envoyée au client

[SERVEUR] Connexion fermée
```

FIGURE 2.1 – Capture fonctionnement côté serveur

```
[SESSION] Session initialisée
[CLIENT] Connexion au serveur 127.0.0.1:5000...
[CLIENT] Connecté au serveur

[CLIENT] Attente du RAND du serveur...
[SESSION] RAND enregistré: 264cabd49f335d16f8a2918fe0d18f76
[CLIENT] RAND reçu: 264cabd49f335d16f8a2918fe0d18f76

[CLIENT] Calcul du SRES...
[CRYPTO] SRES calculé: 7042c7e4
[CRYPTO] SRES calculé: c5314078
[CLIENT] SRES calculé: c5314078
[CLIENT] SRES envoyé au serveur

[CLIENT] Attente du token d'authentification...
[CLIENT] Token reçu: c5314078ea3ecf1c027ec38093c6f721bba3fb18220f425cb6fd08bdd598c57c
[CRYPTO] Token d'authentification calculé: c5314078ea3ecf1c027ec38093c6f721bba3fb18220f425cb6fd08bdd598c57c
[SESSION] Serveur authentifié avec succès
[CLIENT] Serveur authentifié avec succès

[CRYPTO] Clé de session Kc dérivée: 036be1ca7c9683eb5bf1de2e2b6abf6b89f219dea7b08f26c8c8edab182b36e6
[SESSION] Clé de session établie: 036be1ca7c9683eb5bf1de2e2b6abf6b89f219dea7b08f26c8c8edab182b36e6
[CLIENT] Clé de session Kc établie: 036be1ca7c9683eb5bf1de2e2b6abf6b89f219dea7b08f26c8c8edab182b36e6

[CLIENT] Données à envoyer (64 octets): b'a3f7b2c1d9e845607f2c1a3b9d4e6f8a01c3b5d7e9f2a4c6b8d0e1f3a5c7b9d0'
[SESSION] Chiffrement de 64 octets
[CRYPTO] XOR cipher appliqué - Taille: 64 octets
[CLIENT] Données chiffrées: 625887fd1ea4e0da3fc8bb161f5f895bbe942bbd96d1bc44f1acd9ce2e4d0e87335a82f91ea3e7dc3ec8b81c4a5edc5debca7deec281e915a9fd8e9c7a1252d6
[CLIENT] Données envoyées au serveur

[CLIENT] Attente de la réponse du serveur...
[CLIENT] Réponse chiffrée reçue: 420882bf0ff3a38f3ed1ac4b480fcf1fe09d77
[SESSION] Déchiffrement de 19 octets
[CRYPTO] XOR cipher appliqué - Taille: 19 octets
[CLIENT] Réponse déchiffrée: b'Accuse de reception'
```

FIGURE 2.2 – Capture fonctionnement côté client

Conclusion

Ce laboratoire a permis de valider la mise en œuvre complète d'un système d'authentification mutuelle conforme aux principes des réseaux 3G et LTE, confirmant le bon fonctionnement des mécanismes cryptographiques de challenge-réponse bidirectionnels. L'implémentation développée illustre l'évolution significative par rapport au GSM, avec l'introduction d'un token d'authentification permettant au client de vérifier l'identité du serveur, établissant ainsi une session doublement authentifiée avant tout échange de données.

Malgré l'utilisation d'un chiffrement XOR simplifié pour cette simulation, l'architecture mise en place démontre efficacement les principes de l'authentification mutuelle et la dérivation sécurisée de clé de session Kc. Les captures de fonctionnement présentées attestent de la réussite des échanges cryptographiques bidirectionnels et de la validation mutuelle des identités, prévenant ainsi les attaques par usurpation d'identité du réseau.

Enfin, cette étude pratique met en lumière les améliorations apportées par les générations successives de réseaux mobiles, notamment la protection contre les IMSI catchers et les faux relais qui exploitaient l'absence d'authentification du réseau dans le GSM. Elle souligne également la continuité de l'évolution vers le LTE-Advanced et la 5G, qui renforcent encore davantage ces mécanismes avec des algorithmes cryptographiques plus robustes et une meilleure protection de la vie privée. Cette expérience constitue ainsi un maillon essentiel dans la compréhension de l'évolution des architectures de sécurité des communications mobiles.